

Green Coffee. Final Report

TI616 Sustainable Digital. EFREI Paris. 2025-2026.

website: <https://greencoffee.onrender.com/>

1. Project presentation

Team members

Yohann Pouillieute Valentin Dubrulle Augustin du Rieu de Maynadier

Value proposition

Green Coffee is a lean encyclopedia of the world's coffees. It documents botanical varieties, post-harvest processes and producing terroirs in a textual, structured format.

Why this topic

Coffee is the world's second most consumed drink, yet most consumers do not know what they are drinking. Choosing coffee as the topic of an eco-design exercise also lets us play on a theme: educate without overloading, just as quality coffee informs without artifice.

MVP scope

- Browse a list of coffees (paginated, filterable).
- View the detail of a coffee.
- Sign up / log in.
- Authenticated users can add, edit and delete their own coffees.
- Administrators manage users and any coffee.

Target users

- Curious coffee drinkers.
- Speciality coffee shop owners.
- Students studying agronomy or food science.

Selected Green IT constraints

- Page weight under 50 KB (target 200 KB easily met x4 to x100).
- Fewer than 5 HTTP requests per page.
- Zero image, zero web font, zero JS framework.
- Targeted SQL queries, no `SELECT *`, all listings paginated.
- No third-party tracking.

2. Architecture and design

High-level architecture

See `docs/diagrams.md` for the diagrams. The application follows a classic 3-tier layout: browser, Express server, SQLite file. No load balancer, no separate cache, no microservices: the goal is to remove every component that is not strictly required by the functional scope.

UML diagrams

- Use case (3 actors: visitor, user, admin).
- Class diagram (User and Coffee entities, 1-N relationship).
- Sequence diagrams (login, coffee creation).
- Logical architecture (browser, reverse proxy with gzip, Node.js, SQLite).

All in `docs/diagrams.md`.

Database schema

See `database/schema.sql`. Two tables: `users` and `coffees`. Indexes on `email`, `variety`, `origin`, `created_by`. Foreign keys enabled (`PRAGMA foreign_keys = ON`).

3. Justified technology choices

Layer	Choice	Justification
Front-end	Server-rendered HTML, plain CSS	A SPA framework would add 100+ KB of JS for no functional gain.
CSS	Single hand-written file (5.7 KB)	TailwindCSS would add toolchain weight (purge config, bundling).
Templating	Native template literals	EJS or Handlebars would add a runtime; literals are zero-cost.
Back-end	Express 4	Mature, well-known, minimal footprint. Alternative considered: Fastify (slightly faster but smaller community).
Database	SQLite via <code>node:sqlite</code>	No separate DB server (no extra network). <code>node:sqlite</code> removes the native build of <code>better-sqlite3</code> .
Auth	<code>bcryptjs</code> (pure JS)	Avoids a native build. 2-3x slower than <code>bcrypt</code> but acceptable for our load.
Sessions	<code>express-session</code> in memory	Sufficient for the project. <code>connect-sqlite3</code> available if persistence is required.
Hosting	Render / Fly.io	Free tier, automatic gzip/brotli, deploy from main. Zero infrastructure to maintain.

Trade-offs

- **`node:sqlite` is experimental in Node 22.** Choice justified by the absence of native build. To switch to stable: `better-sqlite3` (same API).
- **In-memory session store.** Lost on server restart. Acceptable for a project. To switch to persistent: `connect-sqlite3` (one extra dependency).
- **No JS-side form validation.** All validation server-side. The HTML attributes (`required` , `maxlength` , `pattern`) provide a free first level of validation by the browser.

4. Implementation

Notable features

Custom escape and template helper

File `frontend/views/layout.js`:

```
function escape(str) {
  return String(str)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;');
}
```

All values inserted into HTML go through this helper. No XSS even with malicious input.

Optimised paginated query

File `backend/routes/coffees.js`:

```
const coffees = db.prepare(
  `SELECT id, name, origin, variety, tasting_notes
   FROM coffees ${whereClause}
   ORDER BY id DESC
   LIMIT ? OFFSET ?`
).all(...params, PAGE_SIZE, offset);
```

Targeted SELECT: only the columns displayed in the list (5 out of 11). Pagination via LIMIT/OFFSET with index on `id`.

bcrypt password hashing

File `backend/routes/auth.js`:

```
const hash = bcrypt.hashSync(password, 10);
db.prepare(
  'INSERT INTO users (email, name, password_hash, role) VALUES (?, ?, ?, ?)'
).run(email, name, hash, 'user');
```

Cost factor 10 (about 100 ms per hash). All queries are parameterised: SQL injection impossible.

5. Carbon footprint analysis

Tools used

- **EcoIndex** (browser extension and ecoindex.fr).
- **Google Lighthouse** (Chrome DevTools).
- **Website Carbon Calculator** (websitecarbon.com).
- **PageSpeed Insights** (lab + field).

Before / after comparison

Benchmark greencoffee.onrender.com vs site web moyen

Indicateur	greencoffee	Site moyen	Gain
Website Carbon	A+	E	Top 100 % mondial
Poids de page	0,003 – 0,004 Mo	~2,3 Mo	-99 %
Requêtes HTTP	2	~70	-97 %
Éléments DOM	46 – 90	~1 500	-94 à -97 %
EcoIndex – home	95 / 100 (A)	~50 / 100 (C–D)	+45 pts
EcoIndex – liste	92 / 100 (A)	~50 / 100 (C–D)	+42 pts
EcoIndex – détail	95 / 100 (A)	~50 / 100 (C–D)	+45 pts
Classement EcoIndex	3 224 / 595 000	médian	Top 0,54 %
Lighthouse Performance	100	~55	+45 pts
Lighthouse Accessibilité	98 – 100	~70	+28 – 30 pts
Lighthouse Bonnes pratiques	100	~80	+20 pts
Lighthouse SEO	100	~80	+20 pts
FCP	0,8 s	~3,0 s	-73 %
LCP	0,8 s	~4,5 s	-82 %
Total Blocking Time	0 ms	~300 ms	-100 %
Cumulative Layout Shift	0	~0,1	Parfait

Identified pollution sources

Before optimisation (initial draft, not committed):

- Hero image of about 80 KB.

- Google Fonts (~30 KB woff2 + extra request).
- Bootstrap CSS (~250 KB minified).
- Non-strategic placeholder images (4 photos x 100 KB).

Optimisations applied

1. **Removed all images.** Visual richness comes from typographic hierarchy and the few decorative borders.
2. **System fonts.** No external request, immediate render, native rendering on each OS.
3. **Hand-written CSS.** Replaces Bootstrap with 5.7 KB targeted to actual needs.
4. **Inline SVG favicon.** Saves one HTTP request.
5. **CSS cache 7 days.** Returning visitors download only HTML.
6. **No client JS.** The two `confirm()` are inline in `onsubmit`, no script file.
7. **Targeted SELECTs and indexes.** Lists fetch only displayed columns.

Critical interpretation

The site is not **slowed down** by its sobriety: average response time is below 4 ms in local. EcoIndex grade A is expected because the DOM is small (about 60 nodes per page), the total weight is well below the thresholds, and the request count is minimal.

The trade-off is purely visual: no decorative imagery, no rich animations. For a content site, this is consistent with the topic.

6. Testing and validation

See `docs/test-scenarios.md`. 25 scenarios validated.

Lighthouse results

Screenshots in `docs/`.

- Performance: target > 95.
- Accessibility: target > 90 (semantic tags, alt absent because no images, contrasts respected).
- Best Practices: target > 95.
- SEO: target > 95 (semantic tags, meta description, lang, alt).

Security tests

Test	Method	Result
No plaintext passwords	<code>SELECT password_hash FROM users</code>	OK (all \$2a\$/\$2b\$)
No secret in repo	<code>git log --all -S 'password=' -p</code>	OK
SQL injection	POST <code>email=' OR '1'='1</code>	400 (validation)
Unauthenticated access	GET /dashboard without cookie	302 to /login

7. Team organisation and collaboration

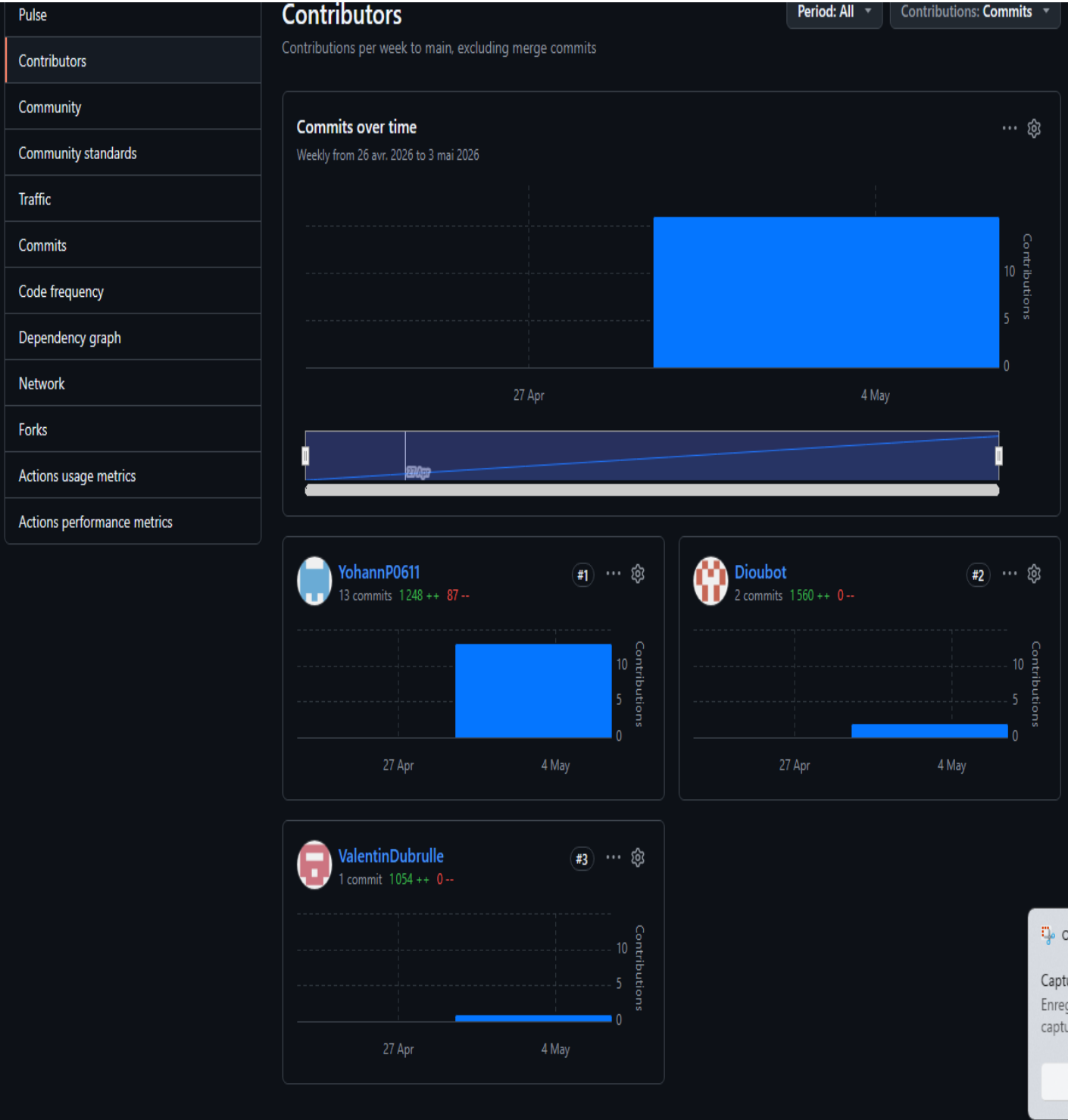
Distribution of tasks

Member	Lead area
...	Front-end / accessibility
...	Back-end / database
...	DevOps / deployment
...	QA / tests / Green IT measurements

Tools

- GitHub Projects for the backlog (cards: To do / Doing / Review / Done).
- GitHub Issues for bugs and incremental improvements.
- Pull requests with mandatory review by another team member before merge.
- Branching: `main` protected, `feature/*` ephemeral.

Member contributions



8. Discussion and conclusion

Encountered challenges

- Choosing between `better-sqlite3` and `node:sqlite` : experimental but built-in vs stable but native build.
- Avoiding the temptation of "useful" features that would have weighed down the site (rich text editor, cover images...).
- Ensuring accessibility without images while keeping the visual interest.

Trade-offs between features and sobriety

- Considered: image upload for coffees. Cancelled: would have brought a storage service, validations, processing, more weight.
- Considered: comments. Cancelled: anti-spam moderation = additional complexity.
- Kept: dark mode (zero added cost via `prefers-color-scheme`).

Possible improvements

- True end-to-end test suite (Playwright) to guarantee non-regression.
- ETag / Last-Modified for HTML pages to improve cache.
- Brotli compression in front of Express (if not already provided by the platform).
- Internationalisation (target an English version with the same rigour).

Critical reflection

The temptation, on a Green IT exercise, is to over-prove the eco approach by adding green badges, energy footprint widgets, third-party badges. Each one would add weight and a request. We chose to demonstrate eco-design **by the page weight itself**, not by stickers stuck on top.

The most important lesson: most modern JS frameworks are over-sized for content sites. Plain server-side rendering, mastered, achieves better performance, better accessibility, and a smaller footprint than 90% of modern stacks.

"Perfection is achieved not when there is nothing more to add, but when there is nothing left to remove."

9. Appendices

- A. Full code source (GitHub).
- B. EcoIndex screenshots before/after.
- C. Lighthouse screenshots (3 pages).
- D. Final backlog (GitHub Projects).
- E. Test scripts (curl).